



Laboratorio: Noticias delictuales, LLM y Obsidian

De texto no estructurado a grafo de conocimiento

Dr. Juan Bekios Calfa

Machine Learning

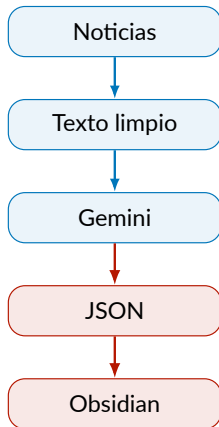
Universidad Católica del Norte

Publicación: 01 Septiembre 2026

Entrega: 29 Septiembre 2026

Idea central del laboratorio

- Los alumnos tomarán noticias delictuales publicadas en Internet.
- Convertirán texto no estructurado en un JSON con entidades, hechos y relaciones.
- La persistencia no será una base de datos tradicional.
- La información final se organizará como una jerarquía de notas Markdown para Obsidian.
- El resultado esperado es un grafo navegable para analizar relaciones entre delitos, personas, lugares y organizaciones.



Pregunta guía

Pregunta de trabajo

¿Cómo transformar noticias delictuales publicadas en Internet en una base de conocimiento estructurada que permita identificar hechos, entidades y relaciones entre delitos?

Restricción metodológica

En este laboratorio no se entrenará un modelo de clustering. Los grupos se formarán por relaciones explícitas: mismo delito, misma persona, misma organización, mismo lugar o combinaciones de entidades compartidas.

Objetivos del laboratorio

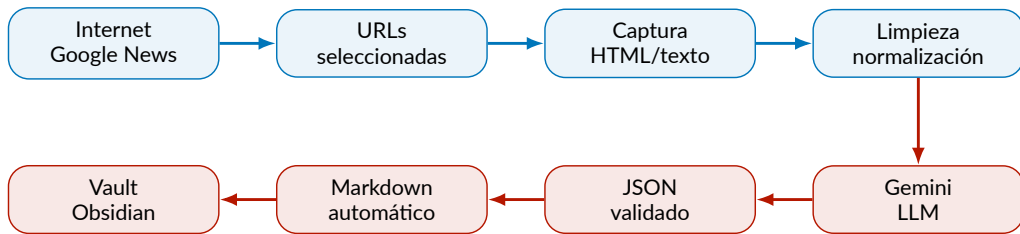
Objetivo general

Construir un pipeline reproducible que capture noticias delictuales, extraiga información relevante con un LLM y genere automáticamente una bóveda de Obsidian para analizar relaciones entre entidades.

Objetivos específicos

- Formular el problema desde Business Understanding.
- Diseñar un modelo de conocimiento: noticias, delitos, personas, organizaciones, lugares y relaciones.
- Capturar noticias desde fuentes abiertas y preparar texto limpio.
- Usar Gemini para extraer información en JSON válido.
- Transformar el JSON en notas Markdown enlazadas.
- Usar Obsidian para explorar relaciones y agrupamientos simples.
- Realizar Data Understanding con estadísticas y visualizaciones básicas.

Arquitectura general



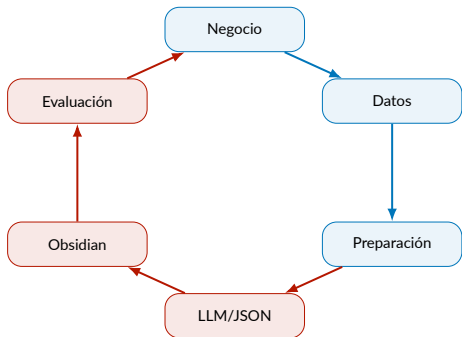
El JSON es un formato intermedio. La persistencia final queda en la jerarquía de archivos de Obsidian.
En azul: etapas ya implementadas en el repositorio. En rojo: módulos que el alumno debe completar.

CRISP-DM adaptado al laboratorio

1. Comprensión del negocio.
2. Comprensión de los datos.
3. Preparación de los datos.
4. Extracción con LLM.
5. Validación del JSON.
6. Generación de conocimiento en Obsidian.
7. Evaluación del resultado y análisis crítico.

Mensaje principal

El laboratorio no comienza con Gemini. Comienza definiendo qué información es útil para analizar un hecho delictual.



Business Understanding

- Se asume el rol de un equipo que apoya a un analista de información delictual.
- El problema no es guardar noticias, sino convertirlas en conocimiento consultable.
- La salida debe permitir preguntas como: ¿qué delitos se repiten?, ¿qué comunas aparecen?, ¿qué personas u organizaciones se mencionan juntas?

Entregable de esta etapa

Una ficha breve con usuario, problema, objetivo, alcance, restricciones éticas y definición de qué se considera una relación útil.

Preguntas para delimitar el alcance

1. ¿Qué tipos de delitos se analizarán?
2. ¿El análisis será nacional, regional o comunal?
3. ¿Qué fuentes de noticias serán permitidas?
4. ¿Cuántas noticias mínimas debe procesar cada grupo?
5. ¿Qué entidades deben extraerse siempre?
6. ¿Cómo se tratarán nombres incompletos o iniciales?
7. ¿Qué información no debe inventar el LLM?
8. ¿Qué significa que dos noticias estén relacionadas?

Modelo de conocimiento

Entidad	Descripción
Noticia	Documento fuente: título, fecha, fuente, URL y resumen.
Delito	Tipo delictual principal o secundario mencionado.
Persona	Persona nombrada o referida en la noticia, con rol explícito.
Organización	Banda, institución, empresa, tribunal, fiscalía u organismo policial.
Lugar	Comuna, ciudad, región, país o punto geográfico.
Objeto	Arma, sustancia, vehículo, dinero u otro elemento incautado.
Relación	Enlace explícito entre entidades: OPERA_EN, INVESTIGADO_POR, DETENIDO_EN.

Principio

Extraer solo lo que aparece explícitamente en la noticia. Si falta información, usar `null` o listas vacías.

JSON esperado

```
{
  "id_noticia": "N001",
  "titulo": "Operativo policial en La Serena",
  "fecha_publicacion": "2026-08-20",
  "fuente": "Medio de prensa",
  "url": "https://...",
  "resumen": "...",
  "delitos": ["Trafico de drogas"],
  "personas": [
    {"nombre": "Juan Perez", "rol": "imputado"}
  ],
  "organizaciones": ["Los X", "Carabineros"],
  "lugares": ["La Serena", "Region de Coquimbo"],
  "objetos": [
    {"tipo": "sustancia", "nombre": "cocaina", "cantidad": null, "unidad": null}
  ],
  "relaciones": [
    {"origen": "Juan Perez", "tipo": "INVESTIGADO_POR", "destino": "Trafico de drogas"},
    {"origen": "Los X", "tipo": "OPERA_EN", "destino": "La Serena"}
  ]
}
```

Adquisición de noticias

- Nivel básico: el repositorio entrega una lista semilla en `data/urls.csv`.
- Nivel intermedio: el grupo busca noticias usando Google News o buscadores generales.
- Nivel avanzado: el grupo ejecuta `python main.py descubrir`, que consulta el RSS de Google News.

Recomendación

Para evitar que el scraping sea el centro del laboratorio, se exige una lista inicial de URLs verificadas. El descubrimiento por RSS **no scrapea** el HTML de `news.google.com`.



Entorno conda

La administración de paquetes se hace **solo con conda**. No hay `requirements.txt`.

```
conda env create -f environment.yml
conda activate lab-noticias-obsidian
python main.py capturar
```

Dependencias

conda-forge: requests, beautifulsoup4, lxml, feedparser, pandas, matplotlib, python-dotenv.
La API de Gemini queda bajo pip: **dentro** de `environment.yml`.

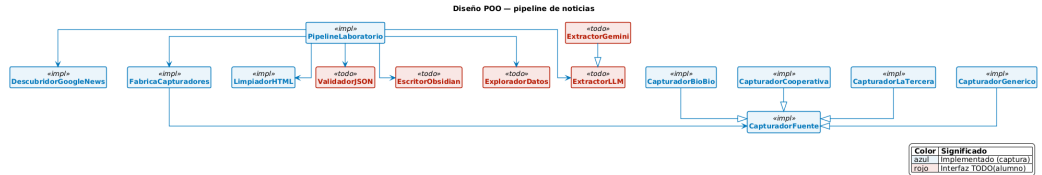
Estructura del proyecto (código modular)

Directorios

```
ML-2026-02-LAB01/  
|-- main.py  
|-- environment.yml  
|-- data/ urls.csv consultas.csv raw/ processed/ json/  
|-- src/  
|   |-- pipeline.py  
|   |-- modelos.py  
|   |-- adquisicion/ (implementado)  
|   |-- limpieza/ (implementado)  
|   |-- extraccion/ (TODO(alumno))  
|   |-- validacion/ (TODO(alumno))  
|   |-- conocimiento/ (TODO(alumno))  
|   |-- analisis/ (TODO(alumno))  
|-- obsidian_vault/  
|-- docs/
```

No se usan notebooks ni scripts sueltos. El orquestador es `main.py`; la lógica vive en clases dentro de `src/`.

Diseño POO



Azul: implementado (captura). Rojo: interfaz **TODO**(alumno).

Archivo de URLs y consultas

```
# data/urls.csv
id_noticia,fuente,url,categoria_busqueda
N001,BioBioChile,https://...,homicidio
N002,Cooperativa,https://...,trafico_drogas
N003,LaTercera,https://...,homicidio

# data/consultas.csv
consulta,categoria_busqueda,limite
homicidio Chile,homicidio,5
trafico de drogas Chile,trafico_drogas,5
```

Trazabilidad

Cada noticia debe mantener su identificador durante todo el pipeline: HTML, texto limpio, JSON y nota Markdown.

Google News: descubrimiento por RSS

```
class DescubridorGoogleNews:
    """No scrapea news.google.com: usa el RSS publico de Chile."""

    def buscar(self, consulta, limite=5):
        params = {"q": consulta, "hl": "es-419",
                  "gl": "CL", "ceid": "CL:es-419"}
        feed = feedparser.parse(GOOGLE_NEWS_RSS + "?" + urlencode(params))
        return list(feed.entries)[:limite]

    def resolver_url_final(self, link_google):
        return self.cliente.url_final(link_google)

    def actualizar_urls_csv(self, consultas=None):
        # Deduplica por URL y asigna id_noticia N00X.
        ...
```

Uso académico

Pausa entre peticiones, timeout y User-Agent identificable. Si un redirect falla, se registra el error y el lote continúa.

Captura OOP: fábrica y adaptadores

```
class CapturadorFuente(ABC):
    def acepta(self, url, fuente=None) -> bool: ...
    def obtener_html(self, url) -> str: ...
    def extraer_cuerpo(self, html) -> str: ...

class FabricaCapturadores:
    def para(self, url, fuente=None) -> CapturadorFuente:
        for capturador in self._especificos: # BioBio, Cooperativa, La Tercera
            if capturador.acepta(url, fuente):
                return capturador
        return self.generico # fallback
```

Fallback

Si el selector CSS del medio no encuentra el artículo, PipelineLaboratorio usa CapturadorGenerico (BeautifulSoup).

Limpieza: clase LimpiadorHTML

```
class LimpiadorHTML:
    ETIQUETAS_RUIDO = ("script", "style", "nav", "footer", "aside")
    LARGO_MINIMO_LINEA = 40

    def limpiar(self, html: str) -> str:
        soup = BeautifulSoup(html, "lxml")
        for etiqueta in soup(list(self.ETIQUETAS_RUIDO)):
            etiqueta.decompose()
        lineas = [ln.strip() for ln in soup.get_text("\n").splitlines()]
        lineas = [ln for ln in lineas if len(ln) > self.LARGO_MINIMO_LINEA]
        return "\n".join(lineas)
```

Cuidado

El grupo debe respetar términos de uso del sitio, evitar sobrecargar servidores y documentar de dónde obtuvo los textos.

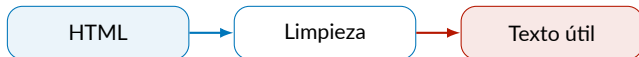
De HTML a texto útil

RAW

HTML completo, menús, publicidad, recomendaciones, scripts, pie de página y contenido duplicado.

Texto limpio

Título, fecha, fuente y cuerpo de la noticia en texto plano, listo para ser enviado al LLM.



Prompt base para Gemini

Analiza la siguiente noticia delictual.

Extrae solamente informacion explicita. No inventes datos.
Devuelve exclusivamente JSON valido, sin explicaciones.

Campos obligatorios:

- id_noticia, titulo, fecha_publicacion, fuente, url, resumen
- delitos, personas (nombre y rol), organizaciones, lugares
- objetos, relaciones (origen, tipo, destino)

Si un dato no aparece, usa null o una lista vacia.

NOTICIA:

{{texto_noticia}}

Idea para discusión

Comparar dos prompts: uno libre y uno con esquema estricto. Evaluar cuál produce JSON más consistente.

Gemini: interfaz para el alumno

```
class ExtractorGemini(ExtractorLLM):
    """TODO(alumno): completar construir_prompt y extraer."""

    def construir_prompt(self, noticia: NoticiaFuente) -> str:
        raise EtapaPendienteAlumno(
            modulo="ExtractorGemini.construir_prompt",
            pista="Prompt estricto + texto de data/processed/{id}.txt",
        )

    def extraer(self, noticia: NoticiaFuente) -> dict:
        # Cargar GEMINI_API_KEY desde .env
        # Llamar a gemini-1.5-flash y guardar data/json/{id}.json
        raise EtapaPendienteAlumno(...)
```

Importante

La clave API no debe subirse a GitHub. Usar variables de entorno o archivo .env excluido por .gitignore.

Validación del JSON: interfaz

```
class ValidadorJSON:
    CAMPOS_OBLIGATORIOS = [
        "id_noticia", "titulo", "fecha_publicacion", "fuente", "url",
        "resumen", "delitos", "personas", "organizaciones",
        "lugares", "objetos", "relaciones"
    ]

    def validar(self, ruta):
        # TODO(alumno): json.loads + campos faltantes + tipos
        raise EtapaPendienteAlumno(...)
```

Contrato de datos

El LLM no es la fuente de verdad. El código debe verificar que su salida cumpla el esquema esperado.

Obsidian como persistencia

- No se usará SQLite, MongoDB ni Neo4j.
- La persistencia final serán archivos Markdown enlazados.
- Cada noticia y cada entidad relevante tendrá su propia nota.
- Las relaciones se expresarán mediante enlaces `[[ . . . ]]` y se visualizarán en el grafo de Obsidian.

Cambio clave

El sistema no guarda filas en una base de datos. Guarda conocimiento como una red de notas conectadas.

Jerarquía del vault

Estructura esperada

```
obsidian_vault/  
|-- 00_Indice.md  
|-- Noticias/  
| |-- N001.md  
| |-- N002.md  
| `-- N003.md  
|-- Delitos/  
| |-- Trafico_de_drogas.md  
| `-- Homicidio.md  
|-- Personas/  
|-- Organizaciones/  
|-- Lugares/  
|-- Objetos/  
`-- Relaciones/
```

La estructura de carpetas es parte del resultado evaluado.

Nota Markdown de una noticia

```
---
id: N001
fecha_publicacion: 2026-08-20
fuente: Medio de prensa
url: https://...
---

# Operativo policial en La Serena

## Resumen
Texto breve generado desde el JSON.

## Delitos
- [[Trafico de drogas]]

## Personas
- [[Juan Perez]] - imputado

## Organizaciones
- [[Los X]]

## Lugares
- [[La Serena]]

## Relaciones
- [[Juan Perez]] -- INVESTIGADO_POR --> [[Trafico de drogas]]
- [[Los X]] -- OPERA_EN --> [[La Serena]]
```

Generar nombres seguros de archivo

```
# Ya implementado en src/conocimiento/utilidades.py
def slugify(text):
    text = unicodedata.normalize("NFKD", text)
    text = "".join(c for c in text if not unicodedata.combining(c))
    text = re.sub(r"[^a-zA-Z0-9]+", "_", text)
    return text.strip("_") or "sin_nombre"

slugify("Tráfico de drogas")          # Trafico_de_drogas
```

Motivo

Los nombres de archivos deben ser estables y consistentes para que los enlaces de Obsidian no se rompan. El alumno debe usar esta función al completar el escritor.

Obsidian: interfaz para el alumno

```
class EscritorVaultObsidian(EscritorObsidian):
    def escribir_noticia(self, data: dict) -> Path:
        # TODO(alumno): plantilla Markdown con [[enlaces]]
        raise EtapaPendienteAlumno(...)

    def escribir_entidades(self, noticias: list[dict]) -> None:
        # TODO(alumno): una nota por delito, persona, lugar, ...
        raise EtapaPendienteAlumno(...)

    def escribir_vault(self, noticias: list[dict]) -> None:
        # TODO(alumno): orquestar noticia + entidades + @@_Indice.md
        raise EtapaPendienteAlumno(...)
```

Persistencia

La salida final es un archivo .md dentro de la bóveda de Obsidian, no un JSON suelto.

Notas de entidades

Además de las noticias, el laboratorio debe generar notas para las entidades.

Ejemplo: Delito

Delitos/Trafico_de_drogas.md

- Noticias relacionadas.
- Personas asociadas.
- Organizaciones mencionadas.
- Lugares donde aparece.

Ejemplo: Persona

Personas/Juan_Perez.md

- Noticias donde aparece.
- Rol observado.
- Delitos asociados.
- Organizaciones relacionadas.

Nota de entidad: delito

Trafico de drogas

Tipo: Delito

Noticias relacionadas

- [[N001]]
- [[N007]]
- [[N015]]

Personas relacionadas

- [[Juan Perez]]
- [[Pedro Soto]]

Organizaciones relacionadas

- [[Los X]]

Lugares

- [[La Serena]]
- [[Coquimbo]]

Construir índices de entidades

```
from collections import defaultdict

indice_delitos = defaultdict(set)
indice_personas = defaultdict(set)
indice_lugares = defaultdict(set)

for data in noticias_json:
    nid = data["id_noticia"]
    for delito in data["delitos"]:
        indice_delitos[delito].add(nid)
    for persona in data["personas"]:
        indice_personas[persona["nombre"]].add(nid)
    for lugar in data["lugares"]:
        indice_lugares[lugar].add(nid)
```

Idea

Los índices permiten escribir automáticamente las notas de delitos, personas y lugares sin usar una base de datos. Incorpórelos en escribir_entidades.

Relaciones y agrupamientos simples

- Un grupo de noticias puede formarse porque comparte el mismo tipo de delito.
- También puede formarse porque comparte una persona, organización, comuna o institución.
- Obsidian permite visualizar esas conexiones sin entrenar un algoritmo de clustering.



Data Understanding

Una vez generado el corpus estructurado, cada grupo debe analizar sus propios datos.

Preguntas mínimas

- ¿Cuántas noticias fueron procesadas?
- ¿Cuántas fallaron?
- ¿Qué fuentes fueron usadas?
- ¿Qué delitos aparecen con mayor frecuencia?
- ¿Qué lugares concentran más menciones?

Calidad de datos

- Porcentaje de campos nulos.
- Noticias duplicadas.
- JSON inválidos.
- Entidades con nombres inconsistentes.
- Relaciones dudosas generadas por el LLM.

Análisis: interfaz ExploradorDatos

```
class ExploradorDatos:
    def noticias_por_fuente(self): ...    # TODO(alumno)
    def delitos_frecuentes(self): ...    # TODO(alumno)
    def lugares_frecuentes(self): ...    # TODO(alumno)
    def campos_faltantes(self): ...      # TODO(alumno)
    def evolucion_temporal(self): ...    # TODO(alumno)

    def ejecutar(self):
        # pandas + matplotlib sobre data/json/*.json
        raise EtapaPendienteAlumno(...)
```

La visualización no es decoración. Debe apoyar la comprensión de los datos y revelar problemas de calidad.

Visualizaciones esperadas

- Noticias por fuente.
- Noticias por tipo de delito.
- Delitos por comuna o región.
- Cantidad de personas u organizaciones por noticia.
- Porcentaje de campos faltantes en el JSON.
- Evolución temporal simple si la fecha está disponible.

Importante

Interprete cada gráfico en el informe: qué dice sobre cobertura, sesgo o calidad.

Control de calidad del LLM

Problema posible	Cómo detectarlo
Inventa entidades	Comparar manualmente una muestra de noticias contra el JSON.
Confunde roles	Revisar si diferencia víctima, imputado, detenido, fiscal, testigo.
Normaliza mal nombres	Crear lista de equivalencias o corregir manualmente.
Produce JSON inválido	Usar <code>json.loads</code> y registrar fallos.
Extrae relaciones dudosas	Exigir que cada relación esté respaldada por el texto.

Muestra de auditoría

Cada grupo debe revisar manualmente al menos 10 noticias procesadas.

Ética y uso responsable

- Las noticias pueden contener acusaciones, investigaciones o información sensible.
- El sistema no debe afirmar culpabilidad si la noticia no lo indica explícitamente.
- Los roles deben registrarse con cuidado: detenido, imputado, acusado, condenado, víctima o testigo no son equivalentes.
- Se debe citar la fuente original y conservar la URL.
- El análisis es académico y exploratorio.

Entregables

- Repositorio GitHub del laboratorio.
- Código modular en `src/` y orquestador `main.py` (sin notebooks).
- Carpeta `data/` con URLs, texto procesado y JSON generado.
- Carpeta `obsidian_vault/` funcionando en Obsidian.
- Visualizaciones de Data Understanding.
- Informe breve con problema, metodología, resultados, errores del LLM y análisis de relaciones.

Pauta de evaluación propuesta

Criterio	%
Business Understanding y alcance del problema	10
Diseño del modelo de conocimiento y JSON	15
Adquisición y limpieza de noticias	15
Extracción con Gemini y validación del JSON	20
Generación automática del vault de Obsidian	20
Data Understanding y visualizaciones	10
Análisis crítico, ética y reproducibilidad	10

Errores comunes

- Guardar solo el JSON y no generar notas enlazadas.
- Usar Obsidian como carpeta de archivos, pero sin enlaces `[[...]]`.
- Pedirle al LLM análisis libre en lugar de un JSON con esquema definido.
- No validar la salida del LLM.
- Mezclar datos inventados con datos extraídos explícitamente.
- No mantener trazabilidad entre URL, texto, JSON y Markdown.
- Presentar gráficos sin interpretar qué dicen sobre los datos.
- Entregar notebooks o scripts sueltos en lugar de clases en `src/`.

Resultados esperados de aprendizaje

Al finalizar el laboratorio, el estudiante debería ser capaz de:

- Formular un problema de análisis de información no estructurada.
- Diseñar un esquema JSON para extracción de información.
- Preparar texto de noticias para ser procesado por un LLM.
- Validar respuestas generadas por un modelo generativo.
- Convertir información estructurada en una base de conocimiento Markdown.
- Usar Obsidian para explorar relaciones y patrones simples.
- Evaluar críticamente la calidad de los datos extraídos.

Secuencia sugerida de trabajo

1. Definir alcance: delitos, región, fuentes y tamaño mínimo del corpus.
2. Crear el entorno: `conda env create -f environment.yml`.
3. Completar o ampliar `data/consultas.csv` y `data/urls.csv`.
4. Ejecutar `python main.py descubrir` y `python main.py capturar`.
5. Completar `ExtractorGemini` y el validador JSON.
6. Completar `EscritorVaultObsidian` y abrir el vault en Obsidian.
7. Corregir nombres inconsistentes.
8. Completar `ExploradorDatos` y el análisis crítico.

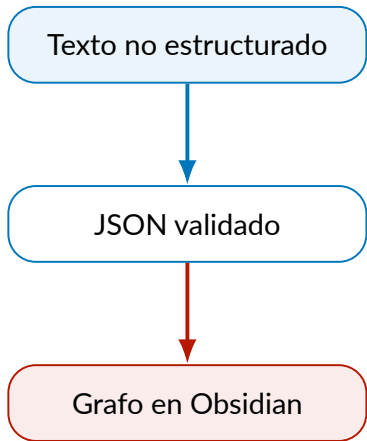
Comandos del orquestador

```
python main.py descubrir    # Google News RSS -> data/urls.csv
python main.py capturar     # URLs -> data/raw + data/processed
python main.py extraer      # TODO(alumno) Gemini
python main.py obsidian     # TODO(alumno) vault Markdown
python main.py analizar     # TODO(alumno) Data Understanding
python main.py pipeline     # descubrir + capturar; avisa pendientes
```

Si una etapa **TODO(alumno)** se invoca sin completar, el programa imprime una pista y **no falla en silencio**.

Cierre del laboratorio

- El valor del laboratorio está en el pipeline completo.
- El LLM ayuda a estructurar, pero no reemplaza la validación humana.
- Obsidian permite convertir datos extraídos en conocimiento navegable.
- Las relaciones emergen desde entidades compartidas, no desde un algoritmo de clustering.



¿Preguntas?